Matplotlib Cheat Sheet

```
import matplotlib.pyplot as plt
```

Customize Plot

Set default values for fontsize and linewidth

```
plt.rcParams["font.size"] = "13"
plt.rcParams["lines.linewidth"] = "2"
```

Title and axis labels

```
plt.title("Heat Equation", fontweight="bold")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
```

Legend

```
plt.legend(loc="right",
        facecolor=(0.9, 0.9, 0.9))
```

```
ax = plt.gca()
ax.set_aspect("equal")    # Don't stretch the plot
plt.grid(b=True)          # enable grid
```

Move axis to origin

```
ax.spines["bottom"].set_position("zero")
ax.spines["left"].set_position("zero")
```

Box off

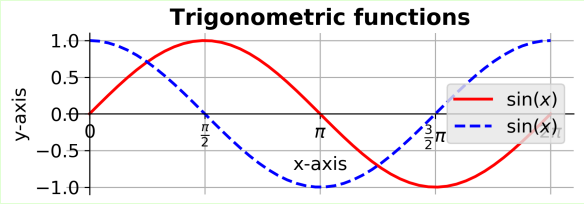
```
ax.spines["top"].set_color("none")
ax.spines["right"].set_color("none")
```

Set x- and y-ticks, as well as labels

```
plt.xticks([0, np.pi, 2*np.pi])
ax.set_xticklabels([r'$0$', r'$\pi$', r'$2\pi$'])
```

Save plot to a png-file, show result:

```
fig.set_size_inches(19.2, 10.8)
plt.savefig("plot_python.png", dpi=300)
plt.show()
```



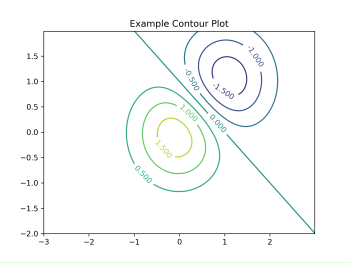
Plot Types

Simple plots:

```
plt.plot(x, y, "xb-", label=r'$f(x)$' )
plt.scatter(x, y, c=None, marker=None)
```

Mathematical plots:

```
plt.contourf([X, Y], Z, [levels])
plt.quiver(X, Y, U, V, C)
plt.streamplot(X, Y, U, V, color=C)
ax.plot_surface(X, Y, Z)
ax.plot_wireframe(X, Y, Z)
```



Diagrams and Data Plots:

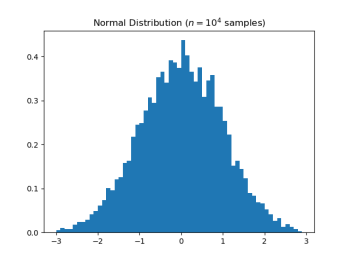
```
plt.bar(x, height, width=0.8)
plt.boxplot(data)
plt.errorbar(x, y, yerr=None, xerr=None)
```

Pie chart:

```
labels = np.array(["txtA", "txtB", "txtC"])
explode = 0.1 * np.array([1, 0, 1 ])
C = np.array(["cyan", "green", "yellow"])

plt.pie(x, explode=explode, labels=labels, col
```

Histogram:

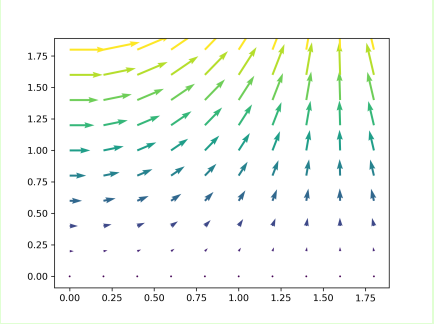
```
# Draw 10^4 random samples, mu=0, sigma=1
data = np.random.normal(0, 1, 10_000)
bins = np.arange(-3, 3, 0.1)

# Normalised histogram with density=True
plt.hist(data, bins, density=True)
```



Quiver-plot

Plot vector fields via `plt.quiver()`:

```
# Create meshgrid
X = np.arange(0, 2, 0.2)
Y = X
X, Y = np.meshgrid(X, Y)

# Define function and color
U = np.cos(X) * Y
V = np.sin(X) * Y
C = np.sqrt(U**2 + V**2)

plt.quiver(X, Y, U, V, C,
                 units="xy")
```

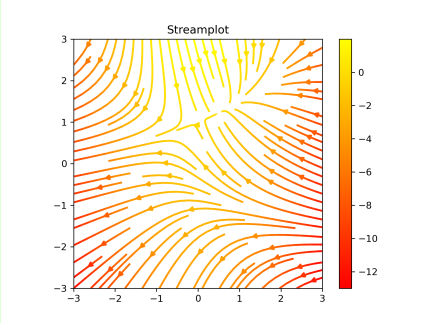


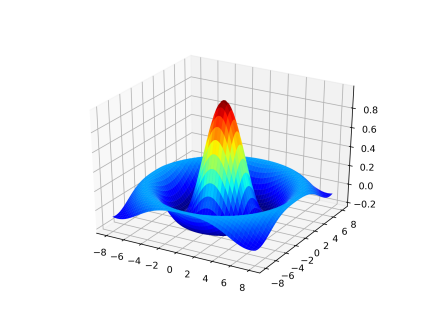
Streamplot

```
Y, X = np.mgrid[-3:3:100j, -3:3:100j]
U = -1 - X**2 + Y
V = 1 + X - Y**2
speed = np.sqrt(U**2 + V**2)

strm = ax1.streamplot(X, Y, U, V, color=U, linewidth=2, cmap='autumn')
fig.colorbar(strm.lines)
```

3D-Surface

```
fig = plt.figure()
ax = fig.gca(projection='3d')

# Create data
X = Y = np.linspace(-8, 8, 100)
X, Y = np.meshgrid(X, Y)
R = np.sqrt(X**2 + Y**2) + 2e-16
Z = np.sin(R) / R

surf = ax.plot_surface(X, Y, Z, cmap="jet", linewidth=0)
```





NumPy and SciPy

Construct and Access Arrays:

```
A = column_stack( (u,v) )
A = row_stack( (u,v) )
M = hstack( (A,B) )
M = vstack( (A,B) )
```

Neat trick:

```
a = np.r_[3,[0]*5,-1:1:10j]
```

Condition number: $\mathbf{cond}(A) := \|A\| \, \|A^{-1}\|$

```
np.linalg.cond(A)
```

Rank:

```
np.linalg.matrix_rank(A)
```

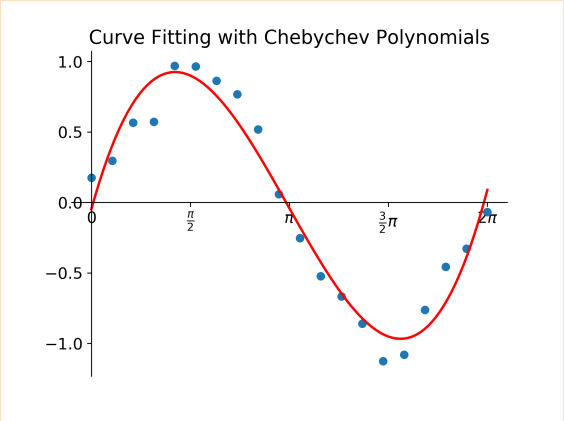
Matrix functions:

```
np.linalg.matrix_power(A, n)     # A^n
linalg.expm(A)                   # matrix exponential
```

Vandermonde-Matrix:

$$V = \mathbf{vander}(\vec{x}) = \begin{pmatrix} x_1^N & \cdots & x_1^2 & x_1 & 1 \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ x_n^N & \cdots & x_n^2 & x_n & 1 \end{pmatrix}$$

```
V = np.vander(x, N)
```



np.polynomial

```
from numpy.polynomial import Polynomial as P
```

$$p(x) = \sum_{k=1}^{n} a_k x^k$$

```
p = P( [a_0, a_1, ..., a_n] )   # Definition of p
p = P.fromroots([1, 2, 3])      # Construct from roots
```

Specify domain and window:

```
p = P( array, domain=[0, 1], window=[-1, 1] )
p.domain = np.array( [a, b] )
p.window = np.array( [a, b] )
```

Integral and Derivative:

```
p.integ()
p.deriv()
```

Polynomial Curve fitting:

```
p = P.fit(x, y, n)
```

SciPy.linalg

```
from scipy import linalg
```

Norm of a vector or matrix:

$$\|A\|_2 = \sqrt{\rho(A^* A)}, \qquad \|A\|_F = \sum_{i,j} |a_{ij}|$$

```
linalg.norm(A)                  # Euclidean/Frobenius
linalg.norm(v, ord=np.inf)      # row-sum norm
```

Moore-Penrose pseudo-inverse:

$$A^\dagger = (A^* A)^{-1} A^*$$

```
linalg.piv(A)
```

Solve Linear System of Equations $A\vec{x} = \vec{b}$

```
linalg.inv(A)           # A^(-1)
x = linalg.solve(A, b)
```

Special function for triangular matrices:

```
x = linalg.solve_triangular(A, b, lower=False)
```

Determinant:

$$|A| = \sum_j (-1)^{i+j} a_{ij} \, |A_{ij}|$$

```
linalg.det(A)
```

Eigenvectors and Eigenvalues:

$$A\vec{v} = \lambda\vec{v}$$

```
lamda = linalg.eigvals(A)       # just eigenvalues

lamda, v = linalg.eig(A)
l1, l2 = lamda          # Eigenvalues
v1 = v[:,0]             # 1st Eigenvector
```

Special functions to calculate Eigenvalues:

`linalg.eigh(A)`	hermitian matrix
`linalg.eigvalsh(A)`	
`linalg.eigh_tridiagonal(A)`	tridiagonal matrix
`linalg.eigvalsh_tridiagonal(A)`	

Schur-Decomposition:

$\forall A \in \mathbb{R}^{n\times n}$, $\sigma(A) \subseteq \mathbb{R} \, \exists$ **orthogonal** U such that:

$$U^* A U = R$$

```
R, U = linalg.schur(A)
```

Singular-Value-Decomposition:

$A \in \mathbb{C}^{n\times n}$ hermitic $\implies \sigma(A) \subseteq \mathbb{R}$

It follows from Schur that:

$$U^* A U = \mathbf{diag}\{\lambda_1, \ldots, \lambda_n\}$$

Applied to $A = M^* M$ we have (if M regular):

$$M = U \Sigma V^{\mathrm{H}}, \quad \Sigma = \mathbf{diag}\{\sigma_1, \ldots, \sigma_n\} > 0$$

```
M, N = A.shape
U, s, Vh = linalg.svd(A)
Sig = linalg.diagsvd(s, M, N)
U @ Sig @ Vh
```

LU- and QR-Decomposition:

$$A = PLU, \quad A = QR$$

```
P, L, U = linalg.lu(A)

# Solve Linear System of Equations:
LU, P = linalg.lu_factor(A)
x = linalg.lu_solve( (LU, P), b)
```

Upper- and lower-trangular matrix:

```
L = linalg.triu(LU, -1) + np.identity(3)
U = linalg.triu(LU)
```

```
Q, R = linalg.qr(A)
```

SciPy.stats

```
from scipy import stats
```

Define a probability distribution:

```
rv = stats.norm(loc=5, scale=2.5)
rv = stats.beta(a=1, b=2)
```

Obtain Statistical Data:

```
rv.mean()
rv.var()
rv.median()
rv.expspect()
rv.stats()      # Mean and Variance
```

Common methods for continuous RVs:

`rv.rvs(size=n)`	Random Variates
`rv.pdf(x)`	Probability Density Function
`rv.cdf(x)`	Cumulative Distribution Function
`rv.ppf(q)`	Inverse CDF

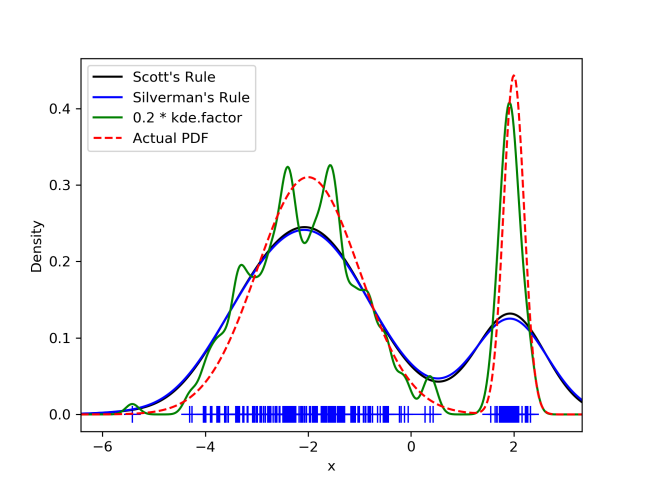
Maximum-Likelihood Estimation:

```
(loc, scale) = stats.norm.fit(data)
```

Kernel density estimation:

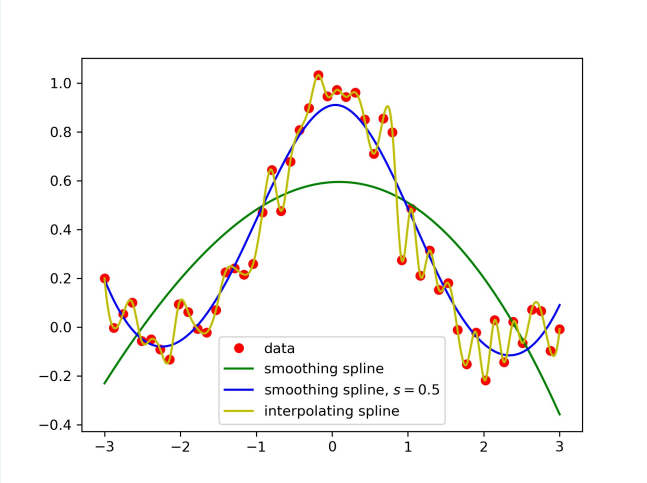
$$\hat{f}(t) = \frac{1}{nh}\sum_{i=1}^{n} k\left(\frac{t-x_i}{h}\right), \quad k(t) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{1}{2}t^2\right)$$

```
kde1 = stats.gaussian_kde(x)
kde2 = stats.gaussian_kde(x, bw_method="silverman")
kde3 = stats.gaussian_kde(x, bw_method=0.2*kde.factor)

plt.plot(x, kde1(x), "k-", label=r'Scotts Rule' )
```

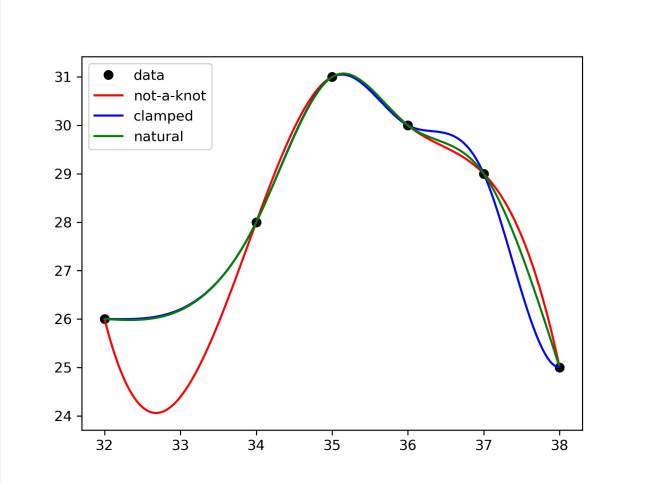


SciPy.interpolate

```
from scipy.interpolate import UnivariateSpline

x = np.linspace(-3, 3, 50)
y = np.exp(-x**2) + 0.1 * np.random.randn(50)
plt.plot(x, y, 'ro', label='data')

# Create UnivariateSpline object, smoothing=0.5
spl = UnivariateSpline(x, y, k=3, s=0.5)
xs = np.linspace(-3, 3, 1000)
plt.plot(xs, spl(xs), 'b-', label='smoothing spline')
```



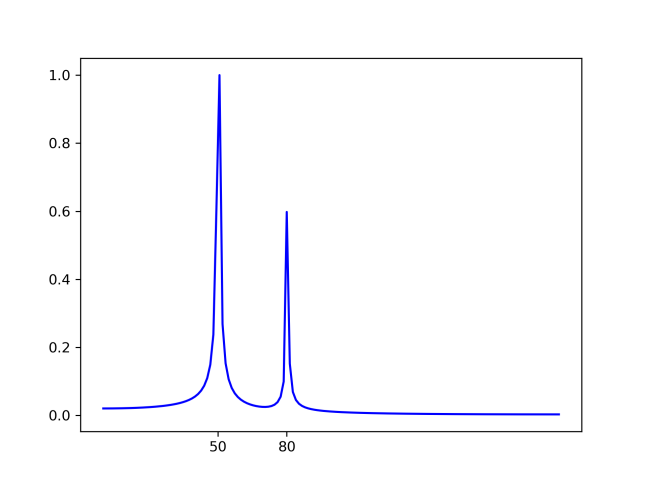

```
from scipy.interpolate import CubicSpline
cs = CubicSpline(x, y)
plt.plot(xs, cs(xs), 'r-', label='not-a-knot')

cs = CubicSpline(x, y, bc_type='natural')
plt.plot(xs, cs(xs), 'g-', label='natural')
```



SciPy.fft

$$y_k = \sum_{n=0}^{N-1} e^{-2\pi i \frac{kn}{N}} x_n$$

```
from scipy.fft import *
from scipy.signal import blackman

N = 300      # number of sample points
T = 1/400    # Sample Rate
x = np.linspace(0, N*T, N)
y = np.sin(50*2*np.pi*x) + 0.5*np.sin(80*2*np.pi*x)
y *= blackman(N)        # Apply filter

xf = fftfreq(N, T)
yf = np.abs(fft(y))

plt.plot(xf, yf, ".b-")
```



SciPy.integrate

```
from scipy.integrate import quad, solve_ivp
```

$$\int_a^b f(x)\,\mathrm{d}x, \qquad \begin{cases} \mathbf{y}'(t) = f(t,\mathbf{y}), & t \in I \\ \mathbf{y}(0) = y_0 \end{cases}$$

```
# 1d-integration:
f = lambda x: x**2
integrate.quad(f, a, b)
```

```
# Solve initial value problem
f = lambda t, y: -y
ivl = (0, 10)
y0 = 1

sol = solve_ivp(f, ivl, y0, method="RK45", t_eval=None)
sol.y   # Solution
```